

# 通用任务

## 5. C++

### 1. 指针是什么？引用是什么？值传参与引用传参有什么不同？

- 指针实际上就是内存中的地址
- 引用实际上是通过内存直接访问变量，从而不新定义一个变量来存储值
- 不同
  - 值传参会在栈中新定义一个变量来存储传入的参数，会产生额外的资源占用，并且在函数结束后，栈上的所有变量都会被丢弃。值传参不会对外界变量产生影响。
  - 引用传参不会额外定义变量，而是直接通过访问内存获得传入变量的值。因此资源占用少，且能够在函数内对传入的变量产生影响

### 2. 在C语言中类和结构体有什么不同？在C++中类和结构体有什么不同？

- C语言没有类
- C++中类和结构体其实也没有什么太大的区别，但是类的成员可见性默认为 `private` 而结构体中的成员可见性默认为 `public`

### 3. C++有哪几种类型转换函数？分别有什么用？

- C 风格类型转换
  - 直接通过在变量前置 `(<type>)`
- C++ 类型转换
  - `static_cast` - 做静态转换
  - `dynamic_cast` - 在运行时做检查，可以转换则转换，不能则返回空指针
  - `const_cast` - 用于更改 `const` 关键词对变量的修饰
  - `reinterpret_cast` - 通过转换指针类型重新解释存储的内容的含义

### 4. auto是什么？有什么用？

- `auto` 是一个关键字
  - 自动设置变量类型
  - 结构绑定

### 5. 基于范围的for循环是什么？有什么用？

- `for (auto/<type> value : values)`
- 在不知大小的情况下直接遍历所有变量知道结束
- 通过引用可以直接修改特定内容
- 简化代码

## 6. 为什么使用#include <xxx.h>就能访问到xxx.h中定义的函数，即#include的原理是什么？

- `<Ctrl-c>` + `<Ctrl-v>`

## 7. 为什么不建议在头文件中使用using namespace？

- 因为头文件经常会被其他文件包含，并且重重叠叠。如果某一个头文件在不知情的情况下被包含到一个不能使用该命名的文件中，那么会出大事

## 8. 智能指针与普通指针有什么不同？有哪几种智能指针？分别有什么特点？使用

`new int` 与使用 `make_shared<int>` 为 `shared_ptr<int>` 初始化有什么不同？

- 智能指针能够自动销毁，而普通指针需要手动 `delete`
- 分别有
  - `std::unique_ptr` - 能够在一个作用域结束之后自动销毁该指针，而该指针只能存在一份，不能被复制
  - `std::shared_ptr` - 在所有复制份都被销毁后才彻底销毁该指针，和Linux的硬链接一样
  - `std::weak_ptr` - 复制一份 `shared_ptr` 而不增加其复制份的计数
- `shared_ptr` 需要一个额外的内存块用于存储复制份的计数，使用 `new int` 会将目标内存块和用于计数的内存块分开分配，而 `make_shared <int>` 则是会将这两个分配步骤合在一起，保证了内存的连续性以及分配的速度

## 9. this有什么作用？static有几种用法？类内静态函数的存储位置跟类内普通函数的存储位置一样吗？类内静态函数的实现中能使用this吗？为什么？

- `this` 指针是指向当前实例的指针，可以用于获取当前目标的成员
- `static` 用法：
  - 用于在文件与文件之间，函数与函数之间传递信息
  - 在类中的静态成员可以通过类名直接访问

## 10. 什么是继承？什么是多态？什么是虚函数？什么是纯虚函数？

- 继承就是从父类中复制黏贴代码
- 多态是指不同的子类对同一父类中的某一方法有不同实现
- 虚函数能通过虚函数表实现子类对父类成员函数的重写，实现通过父类指针正确调用子类方法
- 纯虚函数硬性要求子类对父类中某一方法重写，其余同虚函数一致

## 11. 为什么父类指针能指向子类对象但是子类指针不能指向父类对象？如果能指向父类对象的话会发生什么问题？

- 继承的实际上就是复制黏贴，所以父类的代码在前，在初始化的时候，对于父类中的成员的内存分配也在前。当通过父类指针指向子类对象的时候，实际上是截取了子类中和父类一致的内容，因此

通过父类的指针和虚函数可以正确调用子类中的重写方法。

- 而子类指针若指向父类，则会导致内存占用违法，对不该属于父类的那一块内存进行操作，可能会寄。

## 12. 什么是虚析构函数？为什么需要有虚析构函数？什么是虚函数表？结合虚函数表猜测虚析构函数的实现方法；

- 虚析构函数和虚函数有一点不同，要求子类要重写析构函数
- 虚析构函数保证了某一指针所指的实例能够正确调用子类的析构函数，防止子类中的一些在堆上分配的变量没有被销毁
- 虚函数表是存储了类及其子类中所有虚函数的指针的容器，能够通过该些函数指针让一个对象正确调用虚函数
- 父类虚析构函数通过虚函数表获取子类中的虚析构函数，并在父类的析构函数调用前，先调用子类的析构函数

## 13. 什么是左值？什么是右值？

- 左值是在内存中特定分配了一块地址，有一个对应的变量的值
- 右值是指程序中出现的临时值(还未赋值给变量的值)，但在内存中内有自己的地址

# 6. OpenCV

## 1. 腐蚀和膨胀的原理是什么？开运算和闭运算是什么？和只使用膨胀或只使用腐蚀相比有什么优点？

- 腐蚀和膨胀的原理是对图像进行卷积
- 开运算和闭运算实际上是对图像先后进行腐蚀和膨胀。开运算先腐蚀后膨胀，用于除去目标之外的小噪点，并保持目标不变。闭运算先膨胀后腐蚀，用于弥补目标内部的缺陷
- 能够实现对图像的噪点/缺陷的消除而保留重要信息

## 2. 均值滤波和中值滤波的原理是什么？椒盐噪声和高斯噪声是什么？分别应该怎么处理这两种噪声？

- 均值滤波计算卷积核下图像像素值的均值，而中值滤波则是计算卷积核下图像像素的中值
- 椒盐噪声是由信号脉冲引起的在图像上随机出现的纯黑和纯白的像素点。而高斯噪声则是符合正态分布曲线的噪声
- 椒盐噪声一般通过中值滤波来处理，而高斯噪声一般通过均值滤波来处理

## 3. 什么是bgr？什么是hsv？

- bgr是 blue, green, red 颜色空间，通过这三种颜色的成分组合产生颜色
- hsv是 hue, saturation, value 颜色空间，通过色相，饱和度和明度来组成颜色

#### 4. 什么是直方图？什么是二值化？有哪些自适应二值化方法？其原理是什么？

- 图像直方图是用来表现图像中亮度分布的直方图，给出的是图像中某个亮度或者某个范围亮度下共有几个像素，即统计一幅图某个亮度像素数量
- 二值化是指通过特定的阈值将图像转化成只有0和255像素值，即只有纯黑和纯白的图
- 自适应二值化方法分为全局和局部
  - 全局
    - OTSU
      - 将二值化后的图像分为前景和背景，其中，前景所占比例为  $\omega_1$ ，其像素灰度均值为  $\mu_1$ ，背景所占比例为  $\omega_2$ ，其像素灰度均值为  $\mu_2$ ，图像的整体灰度均值为  $\mu$ ，则其类间方差为  $g = \omega_1(\mu_1 - \mu)^2 + \omega_2(\mu_2 - \mu)^2$ ，通过遍历所给的阈值，使得  $g$  最大的阈值便是自适应阈值
    - 三角形法
      - 对直方图的最高点和最低点连线，并找到在这区间内与该连线距离最大的点，该点所在的阈值就是自适应阈值
    - 迭代法
      - 通过设定初始阈值为全图灰度均值，对其二值化之后的图像分别计算前景和背景的灰度均值，并计算两均值的均值。若均值的均值与设定的阈值相等，则该阈值就是自适应阈值，否则，令新阈值为均值的均值，重复计算，知道二者相等
  - 局部
    - 通过比较某一像素点与周围几个像素灰度的均值或高斯加权均值，若大于计算结果，则设定该点的值为最大值，否则为最小值

#### 5. findContours是什么作用？其每个传参是什么含义？其中hierarchy里面装着什么？

- `findContours` 用于获取图像中的轮廓
- `void findContours(cv::InputArray image, cv::OutputArrayOfArrays contours, cv::OutputArray hierarchy, int mode, int method, cv::Point offset = cv::Point())`
- `cv::InputArray image` - 输入图像
- `cv::OutputArrayOfArrays contours` - 用于存储轮廓点集的容器，其类型为 `std::vector<std::vector<cv::Point>>`
- `cv::OutputArray hierarchy` - 用于存储轮廓之间的层次结构，与 `contours` 中的轮廓一一对应，其类型为 `std::vector<cv::Vec4i>`。其中，`cv::Vec4i` 为 `Vec<int, 4>`，是一个包含4个 `int` 的容器，分别存储了该轮廓下一轮廓，上一轮廓，第一个子轮廓和父轮廓在 `contours` 中的索引
- `int mode` - 枚举类型，用于设置寻找轮廓的方法，如外部轮廓，内部轮廓，列举全部轮廓等
- `int method` - 枚举类型，用于设置描述轮廓的方法，一般为 `None` 和 `Simple`，分别是存储所有点和去掉同一直线上多于的点
- `cv::Point offset = cv::Point()` - 用于调整点的整体偏移

## 6. 手机拍摄中的“扫描”功能（或者叫“文档”功能）的原理是什么？仿射变换与透视变换的原理是什么？

- 透视变换。
- 仿射变换是通过获取3个感兴趣点的变化，从而得到旋转矩阵，进而将矩阵应用于整个图像上  
- 其中，变换矩阵为

$$T = A \begin{bmatrix} x \\ y \end{bmatrix} + B = \begin{bmatrix} a_{00}x + a_{01}y + b_0 \\ a_{10}x + a_{11}y + b_1 \end{bmatrix}$$

- 将其写成矩阵乘积的形式，为

$$T = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix}}_{\text{Translation}} \overbrace{\begin{bmatrix} a_{00} & a_{01} & 0 \\ a_{10} & a_{11} & 0 \\ 0 & 0 & 1 \end{bmatrix}}^{\text{Other Transformation}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} & dx \\ a_{10} & a_{11} & dy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- 透视变换也是通过变换矩阵计算得到的。但是透视变换首先将图像从二维空间投影到三维空间并在这个时候应用变换，然后从三维空间投影到二维空间，形成一张图像  
- 其中，从二维到三维的变换为

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a_1x + b_1y + c_1 \\ a_2x + b_2y + c_2 \\ a_3x + b_3y + c_3 \end{bmatrix}$$

-  $a_3, b_3$  代表着透视变换

- 而从三维向二维投影，则需要将对应的坐标除以  $Z$  的值

$$x' = \frac{X}{Z} = \frac{a_1x + b_1y + c_1}{a_3x + b_3y + c_3}$$

$$y' = \frac{Y}{Z} = \frac{a_2x + b_2y + c_2}{a_3x + b_3y + c_3}$$

- 因此，整个变换可以用下面的矩阵表示

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a_{00} & a_{01} & dx \\ a_{10} & a_{11} & dy \\ b_1 & b_2 & b_3 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

## 7. 实践部分的画画功能是怎么实现的？用自己的话来描述一遍。

- 通过 `inRange` 获取画笔笔头的蒙版，通过 `findContours` 找到画笔笔头的形状轮廓。计算轮廓的面积，通过轮廓面积来代表画笔距离摄像头的距离。当距离小于一定值时，在画笔轮廓的中心画圆。可以额外设置一个变量用于存储前后两个点，并将两个点连线，使得画出的笔画更加连续

# 7. CMake

## 1. 解释用OpenCV输出一张红色图片的代码中的CMakeLists.txt每一行语句的含义；

- `cmake_minimum_required (VERSION 3.10)` - 指定 CMake 的最小版本为3.10
- `project (OPENCV_TEST)` - 设置项目的名称为 OPENCV\_TEST
- `find_package (OpenCV REQUIRED)` - 查找 OpenCV 库，`REQUIRED` 要求一定要找到
- `add_executable (main main.cpp)` - 将 `main.cpp` 生成可执行文件 `main`
- `target_link_libraries (main ${OpenCV_LIBS})` - 将 OpenCV 的库链接到 `main` 文件

## 2. 编译有哪几个步骤？分别在做什么事？

- 预编译 - 读取头文件，处理宏定义
- 编译 - 将项目所有源文件翻译成汇编语言
- 汇编 - 将汇编文件编译成二进制文件，生成一个个目标(obj)，一般为 `.o` 文件
- 链接 - 将目标可执行文件及其他所有的库文件按照代码的逻辑顺序链接在一起，生成一个独立的可执行文件，保证程序能够正确调用库文件

## 3. 什么是静态库？什么是动态库？什么是接口库？

- 静态库是在链接过程中融入到目标可执行文件中的库，即集成与可执行文件
- 动态库是在链接过程中不融入到目标可执行文件的库，而是作为外部以来库
- 接口库是指只有声明而没有实现方法的库

## 4. 结合#include的原理，说明\*.h文件是怎么参与编译的；

- 复制黏贴到包含 `.h` 文件的文件中

## 5. target\_link\_library和link\_library有什么不同？

- `target_link_library` 将需要的库文件和一个目标链接
- `link_library` 对整个项目进行链接

# 8. 机器人学

## 1. 什么是旋转向量？什么是旋转矩阵？怎么实现旋转向量跟旋转矩阵的转换？

- 旋转向量是由一个 **旋转轴** 和一个 **旋转角** 来描述旋转的向量。其方向与转轴一致，长度等于旋转角。
- 旋转矩阵就是一个用于描述物体沿着三个转轴进行旋转的矩阵，表示的是瞬时状态，只描述物体旋转之后的姿态，而不描述其变换过程。
- 旋转向量跟旋转矩阵之间的转换通过 **罗德里格斯公式** 实现

## 2. 什么是固定角？什么是欧拉角？欧拉角有顺序吗？

- 固定角是指绕着 固定不动的用于给物体旋转提供坐标系的正交轴 旋转的转角
- 欧拉角是指绕着 建立在物体上的坐标系的正交轴 旋转的转角，其转轴会随着物体的转动而发生方向上的变化

## 3. 什么是万向锁？

- 万向锁的问题实际上是在旋转过程中，某一次旋转导致 有两个万向节共轴。一般来说，当第二次旋转的角度为90度时，就会导致第一次旋转的转轴和第三次旋转的转轴共线，或者说第一次旋转的万向节与第三次旋转的 万向节共面。这种缺失一个自由度的旋转只可能出现在三个转轴不同的情况下。

## 4. 怎么实现旋转矩阵和欧拉角的转换？

- 直接通过三角函数之间的关系消元计算
- 固定角XYZ：

$${}^A_B R_{XYZ}(\gamma, \beta, \alpha) = \begin{bmatrix} c\alpha c\beta & c\alpha s\beta s\gamma - s\alpha c\gamma & c\alpha s\beta c\gamma + s\alpha s\gamma \\ s\alpha c\beta & s\alpha s\beta s\gamma + c\alpha c\gamma & s\alpha s\beta c\gamma - c\alpha s\gamma \\ -s\beta & c\beta s\gamma & c\beta c\gamma \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

如果  $\beta \neq \frac{\pi}{2}$ ，则：

$$\beta = \arctan(-r_{31}, \sqrt{r_{11}^2 + r_{21}^2})$$

$$\alpha = \arctan\left(\frac{r_{21}}{c\beta}, \frac{r_{11}}{c\beta}\right)$$

$$\gamma = \arctan\left(\frac{r_{32}}{c\beta}, \frac{r_{33}}{c\beta}\right)$$

如果  $\beta = \frac{\pi}{2}$ ，那么， $\alpha = 0, \gamma = \arctan(r_{12}, r_{22})$ ；

如果  $\beta = -\frac{\pi}{2}$ ，那么， $\alpha = 0, \gamma = -\arctan(r_{12}, r_{22})$

- 欧拉角ZYX：

$${}^A_B R_{ZYX}(\alpha, \beta, \gamma) = \begin{bmatrix} c\alpha c\beta c\gamma - s\alpha s\gamma & -c\alpha c\beta s\gamma - s\alpha c\gamma & c\alpha s\beta \\ s\alpha c\beta c\gamma + c\alpha s\gamma & -s\alpha c\beta s\gamma + c\alpha c\gamma & s\alpha s\beta \\ -s\beta c\gamma & s\beta s\gamma & c\beta \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

如果  $\beta \neq 0$ ，那么：

$$\beta = \arctan(\sqrt{r_{31}^2 + r_{32}^2}, r_{33})$$

$$\alpha = \arctan\left(\frac{r_{23}}{s\beta}, \frac{r_{13}}{s\beta}\right)$$

$$\gamma = \arctan\left(\frac{r_{32}}{s\beta}, -\frac{r_{31}}{s\beta}\right)$$

如果  $\beta = 0$ ，那么  $\alpha = 0, \gamma = \arctan(-r_{12}, r_{11})$ ；

如果  $\beta = \pi$ ，那么  $\alpha = 0, \gamma = \arctan(r_{12}, -r_{11})$

## 5. 从世界坐标系到像素坐标系的推导过程中出现了哪些坐标系？

- 世界坐标系 -> 相机坐标系 -> 相机坐标系

## 6. 自己推导一遍各个坐标系转换的公式；

- 从相机坐标系转换到世界坐标系：

$$\begin{aligned} \hat{A}P &= {}^A_R \hat{B}P + {}^A\hat{P}_B \\ \begin{bmatrix} \hat{A}P \\ 1 \end{bmatrix} &= \begin{bmatrix} 0 & {}^A_R & & {}^A\hat{P}_B \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \hat{B}P \\ 1 \end{bmatrix} = T \begin{bmatrix} \hat{B}P \\ 1 \end{bmatrix} \end{aligned}$$

- 从世界坐标转换到相机坐标：

$$\begin{bmatrix} \hat{B}P \\ 1 \end{bmatrix} = T^{-1} \begin{bmatrix} \hat{A}P \\ 1 \end{bmatrix}$$

- 从相机坐标系到归一平面：

$$\hat{P}_c = \begin{bmatrix} X/Z \\ Y/Z \\ 1 \end{bmatrix} = \frac{1}{Z} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \frac{1}{Z} \hat{P}$$

- 从归一平面到像素坐标系：

$$\hat{p} = \frac{1}{Z} K \hat{P} = K \hat{P}_c = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X/Z \\ Y/Z \\ 1 \end{bmatrix}$$

## 7. 什么是相机内参矩阵？

- 相机的内参矩阵是指一个用于描述相机内部参数的矩阵：

$$\begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

- 其中， $f_x = \alpha f$ ,  $f_y = \beta f$ ， $f$ 为相机的焦距， $\alpha, \beta$ 为成像平面(实际成像)与像素坐标系(图片)之间的缩放倍数， $c_x, c_y$ 为两系之间原点的偏移值

## 8. 怎么根据像素坐标系的一个点的坐标计算该点与相机光心之间的欧拉角？

- 根据像素坐标系中的一个点  $[u \ v \ 1]^T$ ，我们可以得到  $\frac{1}{Z} K P$ ，其中  $K$  为内参矩阵， $\frac{1}{Z} P$  为不包含深度信息的位置坐标，是归一化后的相对于相机坐标系的坐标，并且离光心之间的距离为1。
- 然后，我们就可以根据简单的三角关系，根据  $X/Z$  与  $Y/Z$  两项来得到相机的 yaw 角  $\alpha = \arctan \frac{X/Z - |c_x|}{1}$  以及经过 yaw 偏转之后的 pitch 角  $\beta = \arctan \frac{Y/Z - |c_y|}{\frac{1}{\cos \alpha}}$

# 9. Git

## 1. 在github或gitee创建一个账号，建立一个仓库

- Gitee : <https://gitee.com/blake-john>
- Github : <https://github.com/Blake-John>

## 2. 什么是本地仓库？什么是远程仓库？什么是工作区？什么是暂存区？

- 本地仓库就是放在本地的实际的仓库
- 远程仓库实际上说法不明确，有两种含义
  - 在本地建立的 `remote`，只是一个代称而已
  - 云端上的存储库
- 工作区就是实际的目录
- 暂存区就是git存放文件记录的一个概念

## 3. 怎么绑定本地仓库与远程仓库？

- `git remote add <name> <url>`

## 4. add，commit，push分别是在干什么？push命令中加上-f有什么用？

- `add` - 用于向暂存区添加文件
- `commit` - 用于提交仓库
- `push` - 向云端推送仓库
- `push -f` - 强制推送，不管差异直接覆盖云端

## 5. pull是什么？fetch是什么？

- `pull` - 将云端的仓库拉下来并自动和本地的仓库合并
- `fetch` - 仅将云端的仓库拉下来，可以存放于一个新的分支中，不合并

## 6. .gitignore 文件有什么用？

- 用于忽略要加入到工作区中的文件

## 7. 什么是分支？什么是节点？

- 分支就是分支
- 节点就是每一次的提交

## 8. reset是什么？soft、hard分别是什么？

- `reset` - 用于重置仓库到某一节点
- `soft` - 只移动 HEAD 指针，及仓库的状态改变，但是工作区不变

- `hard` - 将 HEAD 指针和工作区都回退到某一节点，在这一节点之后的所有信息都会被清除

## 9. merge是什么？rebase是什么？有什么不同？

- `merge` - 用于合并分支
- `rebase` - 用于变基
- `merge` 会将两分支中的文件合并，并形成一个新的节点，而 `rebase` 只是将节点移动到目标节点之后

## 10. 什么是冲突？什么情况下会发生冲突？

- 冲突就是文件同一行内容不一样
- 在别人修改完某一文件的某一行之后传上去，你也修改了，然后你拉下来，然后就冲突

## 11. git有什么用？跟百度网盘有什么区别？git中应该传build文件夹吗？git中应该传大视频吗？

- 用于版本管理，可以记录每一次提交的情况，方便以后出bug或者想在某一版本的基础上使用新的方法开发的时候，能够获得以前写的代码
- 百度网盘就是依托
- 不应该
- 不应该